Zero Knowledge for Flock's BLAKE3 Prover:
Construction, simulation, and concrete bounds

Alexander Lee

Zero-Knowledge Flock Implementation Candidate
Work in progress; not independently reviewed
July 2026

Abstract

Flock's original BLAKE3 prover [1] was not zero knowledge: its transcript exposed deterministic functions of the witness. This paper gives a zero-knowledge variant for exactly two registered implementation profiles: a batch of 256 BLAKE3 instances and a batch of 256 fixed-digest statements

$$y_i = \mathrm{BLAKE3}(x_i), \qquad |x_i| = 64 \text{ bytes}.$$

Honest and simulated transcripts are checked by the same implementation verifier.

The proof has two algebraic steps. First, a field-valued mask makes the PIOP messages independent of the witness after conditioning on the two mask values that are later opened. For the registered shape, failure occurs on the zero set of a nonzero polynomial of degree at most 720. Second, an affine translation of the PCS blinders preserves every opened value and every recursive codeword. These two translations reduce simulation to choosing the Fiat–Shamir challenges first, solving the dependent messages, and programming at most 18 fresh random-oracle points. The simulator receives only the public statement, and its output is accepted by the implementation verifier.

In the classical programmable random-oracle model, the explicit algebraic and programming terms sum to less than $2^{-118.502}$ at a hash-query budget $Q_H = 2^{64}$, before the ordinary 256-bit oracle-collision term. Thus the two registered profiles are computationally zero knowledge under the assumptions stated here. Knowledge security is separate: one L0 term caps the available standalone Fiat–Shamir reduction over $\mathbb{F}_{2^{128}}$ at 55.994 bits for the same query budget, before other knowledge-error terms are added. The stronger 100-bit deployment target is conjectural, not a theorem of this paper. At the certified batch size of 256, the current unoptimized ZK prover is 6.78 times slower than the current non-ZK prover on twelve threads, and its encoded proof is 9.21 times larger.

1 Result and scope

Result in one sentence. For the two parameter sets in Table 1, a proof generated from a valid witness is computationally indistinguishable from a proof generated from the public statement alone, in the classical programmable-random-oracle model.

The fixed-digest relation is

$$\mathcal{R}_{\mathrm{fd}} = \left\{ \big((y_1, \ldots, y_{256}), (x_1, \ldots, x_{256})\big) : y_i = \mathrm{BLAKE3}(x_i),\ |x_i| = 64 \text{ bytes} \right\}.$$

The batch relation uses the same BLAKE3 circuit but has no externally fixed digest list or hash output. Its statement therefore exposes only the batch shape; the prover is free to choose the

batch trace. The fixed-digest profile is the stronger application statement: it proves knowledge of preimages for public digests while hiding those preimages. In both cases the R1CS shape, packing convention, PCS parameters, and circuit digest are part of the registered statement family.

Throughout the paper, *computational zero knowledge* has its standard classical-ROM meaning: for every classical probabilistic polynomial-time distinguisher making at most Q_H oracle queries, the joint view of an honest proof and its oracle answers is computationally indistinguishable from the view produced by a simulator that knows only the statement. The simulator may program points that the distinguisher has not already queried. This is the IOP and Fiat–Shamir setting of Ben-Sasson, Chiesa, and Spooner [2]; the algebraic masking step is in the spirit of zero-knowledge sumcheck [3].

Table 1: The complete implementation scope of the ZK claim.

Family	Public input	Circuit / PCS	Status
BLAKE3 batch	batch size 256	$m = 22$, rate 1/2, batch log 6	computational ZK
BLAKE3 preimage	256 digests	$m = 22$, rate 1/2, batch log 6	computational ZK

The scope is deliberately narrow. The preimage circuit hashes one message of exactly 64 bytes and pins the BLAKE3 IV, counter, block length, and `CHUNK_START`, `CHUNK_END`, and `ROOT` flags. Unknown circuit digests, batch sizes, and PCS parameters are rejected before proving. SHA-256 statements, Keccak statements, longer BLAKE3 messages, hash chains, Merkle-path statements, recursion, side channels, and quantum random-oracle queries are not covered.

The argument has a simple structure:

1. translate the PIOP masks while holding their revealed values fixed;
2. translate the PCS blinders while holding every opening fixed;
3. sample the remaining transcript from the public statement and program only fresh, domain-separated oracle points.

Sections 4–6 prove these steps in that order. Extraction is discussed separately because witness hiding and knowledge soundness are different properties.

Assumption 1 (Classical programmable random oracle)**.** All injectively framed SHA-256 calls are modelled as one random function $\mathcal{H} : \{0,1\}^* \to \{0,1\}^{256}$. The adversary makes classical queries, and the simulator may program a point only before that point is queried. No QROM claim is made.

2 Relation and construction

Let z be the packed binary assignment for the block-diagonal BLAKE3 R1CS, embedded in $\mathbb{F}_{2^{128}}$. The prover establishes

$$(Az) \circ (Bz) = Cz$$

using Flock's zerocheck and lincheck, and opens the committed vector with the ring-switched Ligerito PCS [4, 5]. For the fixed-digest family, the verifier derives an additional packed opening claim from the public digest list. Before the first challenge, the transcript binds that list, the padding rule, the witness layout, the circuit matrices, the parameter set, and a fresh 32-byte proof nonce.

The ZK prover adds four independent mask sources: the zerocheck mask P, the lincheck mask S, and the round-one masks S_c, S_h. The witness and mask commitments use independent hiding randomness, denoted μ and g. Every stream, including the proof nonce, is derived from a per-proof DRBG under a distinct label.

The main change is the zerocheck mask

$$\gamma P(X)Q^\star(X), \qquad Q^\star(X) = q_0 + \sum_j q_j X_j.$$

Here P is uniform over $\mathbb{F}_{2^{128}}$ on a translated subcube of the top $d_{\log} = 12$ variables. The affine polynomial $Q^\star$ is public, protocol-fixed, and nonzero on the Boolean cube. The field-valued mask is essential: the withdrawn Boolean P, Q construction did not support the Schwartz–Zippel argument needed for the registered parameters. Protocol `flock-zk-fv-v3` therefore has no Q commitment, Q opening, final Q evaluation, or Q-randomness stream.

3 One random-oracle boundary

Fiat–Shamir, Merkle commitments, and proof of work use one oracle, but never the same input. Each input begins with a role byte. A Merkle input also binds the wire-protocol identifier, proof nonce, commitment channel, recursion depth, node level, node index, payload length, and payload. A proof-of-work input is

$$\texttt{ROLE\_POW} \parallel \mathit{state} \parallel \mathit{nonce}.$$

The proof nonce is absorbed immediately after the statement and before the first challenge.

This framing is part of the proof, not merely an encoding convention. It lets the extractor identify witness-leaf queries without relying on their order, prevents simulator programming from aliasing transcript or Merkle inputs, and makes proofs from different nonces or commitment channels use different oracle points. Native, recording, and programming backends implement the same frames. When no point is programmed they return identical bytes. Scalar and four-way SIMD implementations are differentially tested against the one-shot definition for every supported tail shape.

4 The PIOP translation

Fix the post-commitment challenge tuple c. Let D be the coefficient space of P. If $n = m - 6$ is the number of masked zerocheck rounds, define

$$R_c : D \longrightarrow \mathbb{F}_{2^{128}}^{2n}$$

to be the map from mask coefficients to the $2n$ round-message coordinates. Let

$$L_c : D \longrightarrow \mathbb{F}_{2^{128}}^{2}$$

map the same coefficients to the two mask values revealed later: the initial masked claim and $P(\rho)$. Once the leakage $L_c(P)$ is fixed, the remaining mask freedom is exactly $\ker L_c$, and the available change in the round messages is $R_c(\ker L_c)$.

Theorem 1 (Conditioned field-mask coverage)**.** *For $m \in \{13, 15, 22\}$, the symbolic certificate establishes, outside the zero set of its selected minor,*

$$\operatorname{rank}\begin{bmatrix} R_c \\ L_c \end{bmatrix} = 2n, \qquad \operatorname{rank} L_c = 2, \qquad \dim R_c(\ker L_c) = 2n - 2.$$

For the registered shape $m = 22$, the selected determinant is a nonzero polynomial of total degree at most 720 in the 32 challenge variables. Hence

$$\Pr_c[\textit{coverage failure}] \le \frac{720}{2^{128}} < 2^{-118.508}.$$

Proof sketch. Let $N = \dim D$. The stacked-rank statement gives

$$\dim(\ker R_c \cap \ker L_c) = N - 2n,$$

while $\operatorname{rank} L_c = 2$ gives $\dim \ker L_c = N - 2$. Applying rank–nullity to R_c restricted to $\ker L_c$ yields

$$\dim R_c(\ker L_c) = (N - 2) - (N - 2n) = 2n - 2.$$

The implementation selects a $2n \times 2n$ minor and evaluates its determinant to a nonzero field element at one explicit challenge tuple. Its determinant is therefore not the zero polynomial. The generic symbolic kernels bound its total degree by 720, so Schwartz–Zippel gives the stated bad-set probability [6, 7]. □

The concrete and symbolic matrices come from the same generic round kernels. The symbolic scalar has three interpretations: evaluation in $\mathbb{F}_{2^{128}}$, conservative multivariate degree bounds, and exact sparse polynomials for small instances. It has no inversion operation, so a challenge-dependent denominator cannot enter the calculation unnoticed. The selected rows, columns, nonzero evaluation, and degree vector are pinned in `s2_mask_coverage.json` and replayed by the certificate suite.

Theorem 1 proves the PIOP masking step. It says nothing by itself about privacy at the PCS opening boundary; that is the next step.

5 The PCS translation

Write the initial combined opening vector as

$$F = (\mu \parallel z) + c\, g,$$

where $c \in \mathbb{F}_{2^{128}}$ is sampled after commitment. Let d be a witness difference that preserves every public opening claim. The aim is to replace z by $z + d$ without changing anything the verifier opens.

Lemma 1 (Opening-preserving translation)**.** *Suppose $c \neq 0$ and the initial-level query system has full row rank. There is an affine bijection on the uniform PCS masks such that replacing z by $z + d$ leaves the queried initial rows and the entire combined vector F unchanged. Consequently every recursive codeword, root, authentication path, sumcheck value, and out-of-domain sample is unchanged.*

Proof. Set

$$\delta g_{\mathrm{top}} = c^{-1} d.$$

Choose $\delta\mu$ to solve the queried initial-level encoding equations, and set

$$\delta g_{\mathrm{lo}} = c^{-1}\delta\mu.$$

Because the field has characteristic two, $d + c\,\delta g_{\mathrm{top}} = 0$ and $\delta\mu + c\,\delta g_{\mathrm{lo}} = 0$. Thus $\delta F = 0$ globally. The defining equations for $\delta\mu$ also hold the separately opened initial rows fixed. Translation by these deltas is a bijection on uniform masks, so it preserves their distribution. □

The registered opening queries 218 initial-level positions; each lane contains 512 independent mask symbols. After conditioning on all opened positions, each additional fresh wide leaf retains at least 16,384 bits of conditional entropy. No recursive entropy charge is needed because Lemma 1 makes the recursive codewords identical. The exceptional event $c = 0$ has probability 2^{-128}.

The proof format contains twelve categories of opening fields. The implementation exhaustively destructures and classifies them, so adding an unclassified field fails compilation. The exact translator is tested against the registered additive NTT and wide interleaving. Its functional manifest and entropy table are pinned in `s3_opening_functionals.json` and `s3_minentropy_table.json`.

6 Simulator and concrete ZK bound

Once Theorem 1 and Lemma 1 are separated, the simulator is direct. It receives a public-statement capability containing the setup and digest list, but no witness type. It constructs an unrelated trace, overwrites the public-output region, samples the terminal zerocheck values, and solves the dependent messages in one pass. The patched trace is not an R1CS witness: running the honest prover on it is rejected. After the simulator programs the required fresh oracle points, however, the implementation verifier accepts the simulated proof. The simulator programs at most 18 points.

The hybrid argument is summarized in Table 2.

Table 2: Game changes and their explicit charges.

Game	Change	Charge
G_0	honest proof	0
G_1	translate the PIOP mask	$720/2^{128}$
G_2	translate μ and g	$1/2^{128}$
G_3	cross the Merkle boundary	$Q_H/2^{16384} + \varepsilon_{\mathrm{RO}}$
G_4	program fresh challenges	$18Q_H/2^{256}$
G_5	solve the terminal messages	$2/2^{128}$

The three algebraic numerators are 720, 1, and 2: the symbolic bad set, $c = 0$, and the two rejected values of the terminal challenge. Thus the part of the bound tracked explicitly by the executable ledger is

$$\varepsilon_{\mathrm{explicit}}(Q_H) = \frac{723}{2^{128}} + \frac{18Q_H}{2^{256}} + \frac{Q_H}{2^{16384}}.$$

At $Q_H = 2^{64}$,

$$-\log_2 \varepsilon_{\mathrm{explicit}}(2^{64}) = 118.502\ldots.$$

The term $\varepsilon_{\mathrm{RO}}$ denotes collision and binding error at the framed 256-bit oracle boundary. It remains symbolic because a full count must include both adversarial queries and the protocol's own Merkle, transcript, and grinding queries. It is not included in the quoted 118.502-bit figure, so that number is the explicit algebraic and programming ledger, not a complete standalone security level.

Theorem 2 (Scoped computational zero knowledge)**.** *Assume the classical programmable random oracle of Assumption 1, the conditioned coverage of Theorem 1, the opening translation of Lemma 1, and freshness of the programmed points. Then both profiles in Table 1 are computationally zero knowledge. For every classical distinguisher making at most Q_H oracle queries,*

$$\mathrm{Adv}_{\mathrm{ZK}} \leq \varepsilon_{\mathrm{explicit}}(Q_H) + \varepsilon_{\mathrm{RO}}.$$

The simulator uses only the public statement, and the verifier is the same implementation verifier used for honest proofs.

This theorem is in the classical programmable-ROM. It is not a QROM theorem, and it does not turn the ideal-oracle argument into a standard-model theorem for SHA-256.

7 Performance

Table 3 compares the certified ZK path with the current non-ZK Flock path at their common supported batch size. Both use the same 256 BLAKE3 compression inputs and the same repository revision. Measurements were taken on a 12-core Apple M4 Pro with 24 GB of memory, using twelve Rayon threads and the best of ten trials after one verified warm-up.

Table 3: Measured cost for a batch of 256 BLAKE3 compressions.

Metric	Non-ZK Flock	Certified ZK-Flock	ZK / non-ZK
Prover time per batch	4.378 ms	29.674 ms	6.78×
Prover throughput	58,474/s	8,627/s	0.148×
Verifier time per batch	9.257 ms	5.306 ms	0.57×
Encoded proof size	265.7 KiB	2,447.9 KiB	9.21×

On one thread, the same harness measured 12.506 ms for non-ZK proving and 44.372 ms for certified ZK proving, a factor of 3.55×. Native scalar BLAKE3 took 58.5 ns per compression on this machine. A native batch of 256 therefore takes about 14.97 μs, giving measured single-thread overheads of about 836× for current non-ZK Flock and 2,965× for the certified ZK path at this small batch.

The original Flock paper [1] reported less than 250× native overhead at the batch size that maximized throughput. It benchmarked powers of two through $2^{18} = 262{,}144$ BLAKE3 compressions and reached its peak single-thread throughput at 2^{16}; the construction did not state a hard maximum batch size. Those figures use much larger batches, different hardware, and an earlier implementation, so they are context rather than a direct baseline for Table 3. In particular, multiplying 250 by 6.78 would not be a measured ZK-Flock overhead.

The certified path is deliberately oriented toward auditability and still uses naive round-pair kernels. The table measures the implementation candidate as it exists; it does not claim that the observed factor is an intrinsic cost of zero knowledge. The matched benchmark is reproduced by:

```
ZK_BENCH_THREADS=12 ZK_BENCH_RUNS=10 scripts/zk-benchmark.sh
```

8 Extraction and simulation extractability

Zero knowledge means that a proof reveals nothing about the witness beyond the truth of the statement. Knowledge soundness asks the complementary question: can a successful prover be used to recover a witness? The two properties use different arguments and have different concrete bounds.

The recording oracle observes the framed leaf queries used to construct the witness commitment. Given a complete valid codeword, the extractor rebuilds the interleaved word, applies the inverse NTT lane by lane, re-encodes the candidate, checks the configured unique-radius condition, and returns the witness half. The system-specific step then reads the 512 message bits of each instance and independently verifies

$$\mathrm{BLAKE3}(x_i) = y_i.$$

Honest recorded commitments yield the true preimages; simulated commitments do not extract.

The executable decoder is an exact-word decoder and candidate validator. It is not a complete Reed–Solomon decoder for every noisy word inside the theoretical radius. The general noisy-word step therefore comes from the Ligerito proximity and unique-decoding argument, not from the current decoder routine.

For simulation extractability, the simulator records

$$(\textit{statement digest}, \textit{proof nonce}, \textit{protocol identifier})$$

for each simulated proof. Extraction refuses every recorded prefix and is attempted only on a fresh one. The implemented property is therefore *fresh-prefix weak simulation extractability*. Same-prefix simulation extractability is not claimed.

9 Knowledge-security profiles

The knowledge ledger contains an interactive L0 fold term

$$\kappa_\Gamma = \frac{257}{2^{128}},$$

or 119.994 bits. The classical Fiat–Shamir reduction multiplies this term by $Q_H + 1$. At $Q_H = 2^{64}$ this term alone becomes

$$-\log_2\big((Q_H + 1)\kappa_\Gamma\big) = 55.994\ldots \text{ bits}.$$

Thus 55.994 bits is a ceiling imposed by one term, not a completed lower guarantee for the whole argument. Every additional positive knowledge-error term can only reduce the final number of provable bits.

The 64-bit loss already occurs in the classical reduction. It comes from guessing which of the adversary's Q_H oracle queries carries the challenge used in the successful proof, which contributes the factor $Q_H + 1$. No quantum adversary is needed for this loss. Thus "classical loss" means that the stated standalone knowledge theorem is already only 55.994 bits against classical attackers. It does not mean that the protocol is post-quantum secure. The paper gives no quantum-random-oracle reduction and therefore makes no knowledge-security claim against quantum adversaries.

The standalone implementation profile must be labelled *100-bit conjectured classical knowledge security*. Its reduction ceiling is 55.994 bits before the remaining terms are added. An unbiased

Table 4: Knowledge claims are separate from the ZK theorem.

Profile	Reduction ceiling	Status
Standalone FS over $\mathbb{F}_{2^{128}}$	at most 55.994 bits at $Q_H = 2^{64}$	deployment label: 100-bit conjectured classical knowledge security
Unbiased post-commitment beacon over $\mathbb{F}_{2^{128}}$	119.994 bits	100-bit theorem; not standalone
Standalone FS with challenge-only $\mathbb{F}_{2^{256}}$	183.994 bits at $Q_H = 2^{64}$	not implemented

post-commitment beacon removes the Fiat–Shamir query loss but changes the protocol model. A challenge-only $\mathbb{F}_{2^{256}}$ profile would restore a wide standalone margin, but the required machinery does not exist in this implementation. Grinding, a small-Q_H convention, generic parallel repetition, and deleting one fold challenge do not repair the standalone bound.

10 Extending the ZK construction

The masking code is reusable, but the ZK claim does not transfer automatically to another hash, circuit, batch size, or PCS shape. A new family such as SHA-256 or Keccak requires all of the following steps:

1. define the exact relation, public inputs, padding, and fixed hash parameters;
2. bind the circuit digest, witness layout, PCS parameters, and complete transcript schema;
3. regenerate the symbolic PIOP map and prove a nonzero determinant with an explicit all-challenge degree bound;
4. prove that the PCS mask translation is solvable for every query set the implementation verifier can derive, and recompute the unopened-leaf entropy;
5. construct a simulator whose API contains only the public statement and whose output is accepted by the unchanged family verifier;
6. enumerate every verifier-visible field and prove the hybrid composition, including random-oracle programming and grinding order;
7. implement the relation-specific extractor and recompute the complete ZK and knowledge-error ledgers; and
8. add a fail-closed certificate entry only after the family-specific negative controls and end-to-end certification pass.

The existing SHA-256 and Keccak encoders satisfy none of these obligations by their existence alone. They remain non-ZK until their own artifacts, simulators, extractors, and certificate entries are reviewed.

11 Reproduction and limitations

The wire protocol is `flock-zk-fv-v3`; proof I/O format 6 and transcript schema 5 bind the nonce, oracle frames, and field-mask schema. The two registered circuit digests are:

```
BLAKE3 batch:
4a398ca2e73d9d1f70611bd6a2a408b0404aabddcf6cf75c29cfa412f72f8423
BLAKE3 fixed digest:
e71bde2b053b1b697412d2d30c2e2e57f7b5f1822484fcbcc15c89ea202890d5
```

The complete evidence gate is:

```
rustup override set stable
scripts/zk-certify.sh
cargo test --release --workspace --features zk,symbolic
cargo clippy --workspace --all-targets --features zk,symbolic -- -D warnings
cd lean && lake build
```

It replays the symbolic artifacts, transcript schema, oracle framing, registered-shape rank gates, simulator, extractor, fixed-digest and batch roundtrips, Lean adapters, knowledge ledger, and the ban on unframed direct SHA-256 calls. Evidence names are compared with script test names in both directions.

The executable evidence is useful because it ties each numerical claim to the implementation. It is not a substitute for a cryptographic proof or for independent review. The remaining limits are:

- The ZK theorem is classical-ROM only; there is no QROM analysis.
- The standalone knowledge reduction over $\mathbb{F}_{2^{128}}$ is capped at 55.994 bits at $Q_H = 2^{64}$ before other terms are added. This is not a completed 100-bit theorem.
- The executable decoder validates candidates but does not implement general arbitrary-error Reed–Solomon decoding.
- The exact PCS translator is exercised on a fixture; a standalone proof for every query set derived by the implementation verifier remains a review obligation.
- Only the two exact registry entries are enabled. Other Flock statement families remain non-ZK.
- Side-channel resistance, recursive composition, and same-prefix simulation extractability are out of scope.
- The construction and proof have not received independent cryptographic review.

Within these limits, the conclusion is precise: for the registered BLAKE3 batch and fixed-digest relations, Flock now has a public-statement-only simulator whose proofs are accepted by the implementation verifier, together with an explicit distribution argument for every witness-dependent part of the transcript. That is the sense in which these Flock profiles are zero knowledge.

References

[1] Succinct Labs. *Flock*. Historical paper at repository commit `73f7202`. `https://github.com/succinctlabs/flock/blob/73f7202/paper/flock-paper.pdf`.

[2] E. Ben-Sasson, A. Chiesa, and N. Spooner. *Interactive Oracle Proofs*. TCC 2016; IACR ePrint 2016/116.

[3] A. Chiesa, M. A. Forbes, and N. Spooner. *A Zero Knowledge Sumcheck and its Applications*. IACR ePrint 2017/305.

[4] A. Novakovic and G. Angeris. *Ligerito: A Small and Concretely Fast Polynomial Commitment Scheme*. IACR ePrint 2025/1187.

[5] B. E. Diamond and J. Posen. *Polylogarithmic Proofs for Multilinears over Binary Towers*. IACR ePrint 2024/504.

[6] J. T. Schwartz. *Fast Probabilistic Algorithms for Verification of Polynomial Identities*. Journal of the ACM 27(4), 701–717, 1980.

[7] R. Zippel. *Probabilistic Algorithms for Sparse Polynomials*. EUROSAM 1979, LNCS 72, 216–226.